Policy Gradients and Variance Reduction

Learning from Experience

1

Learning Objectives

- Explain why **value-based RL** (Q-learning, DQN) is impractical for LLMs
- Derive and interpret the **policy gradient theorem**
- Write the **REINFORCE** update rule and explain its high-variance problem
- Define the **advantage function** and explain why it reduces variance
- Describe the **actor-critic** architecture and the role of **GAE**

2

From Last Time

- We formalized sequential decisions as **MDPs**: $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$
- An LLM is a **policy**: $\pi_\theta(\text{token}|\text{context})$
- The **Bellman equation** gives us the recursive structure of value
- **Today's question**: How do you *optimize* a policy when you can't enumerate states or know transitions?
- **By the end of this lecture**: the exact mathematical technique that lets you train a **billion-parameter transformer** using only a **scalar reward signal**

3

Part 1: The RL Problem

4

The Agent-Environment Loop

- The fundamental RL interaction:

```
repeat forever:
  Agent observes state s_t
  Agent samples action a_t ~ π_θ(·|s_t)
  Environment returns reward r_t
  Environment transitions to s_{t+1}
```

- The agent generates its own training data by interacting with the environment
- No teacher provides the correct answer — only a **scalar reward signal**

5

Supervised Learning vs. Reinforcement Learning

	Supervised Learning	Reinforcement Learning
Feedback	Correct label for each input	Scalar reward (no 'correct answer')
Data	i.i.d. samples	Sequential, correlated, agent-generated
Consequences	None — predictions are independent	Actions affect future states
Core challenge	Generalization	Credit assignment + exploration

6

Why Value-Based Methods Don't Scale

- **Q-learning idea**: learn $Q(s, a)$ — the expected return from taking action a in state s, then acting optimally
- $$Q(s,a) \leftarrow Q(s,a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s,a) \right]$$
- **DQN** (2015): approximate Q with a neural network → solved Atari games
- **But**: requires $\max_{a'} Q(s', a')$ over **all actions**
- With a 100K-token vocabulary → evaluating Q for every possible next token at every step
- **Computationally prohibitive** for language models

7

The Policy-Based Insight

- We have a neural network that already outputs a **probability distribution** over tokens
- We should **optimize that distribution directly**
- No need to compute Q for every action — just sample and learn
- This is the **policy gradient** approach
- Everything from here forward is **policy-based RL** — this is what every lab actually uses

8

Part 2: The Policy Gradient Theorem

9

Interactive Demo: Policy Gradient

🚀 **Interactive Demo:** policy_gradient_demo.html

10

The RL Objective

- Maximize **expected total reward** under the policy:
- $$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}\left[\sum_{t=0}^{T} r_t\right]$$
- Where $\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \ldots)$ is a **trajectory** sampled by running the policy
- We want: $\theta^* = \arg\max_\theta J(\theta)$
- **Strategy**: gradient ascent — compute $\nabla_\theta J(\theta)$ and step uphill

11

The Fundamental Problem

- We **cannot** differentiate through the environment:
- The reward function isn't a differentiable function of θ
- The sampling process ($a_t \sim \pi_\theta$) isn't differentiable
- Standard **backpropagation doesn't apply**
- We need a clever mathematical trick...

12

The Log-Derivative Trick

- For any distribution $p_\theta(x)$ and function $f(x)$:
- $$\nabla_\theta \, \mathbb{E}_{x\sim p_\theta}[f(x)] = \mathbb{E}_{x\sim p_\theta}\left[f(x) \cdot \nabla_\theta \log p_\theta(x)\right]$$
- **Derivation** (three lines):
- $$\nabla_\theta \mathbb{E}[f(x)] = \nabla_\theta \int p_\theta(x) f(x)\, dx = \int f(x)\, \nabla_\theta p_\theta(x)\, dx$$
- $$= \int f(x)\, p_\theta(x)\, \frac{\nabla_\theta p_\theta(x)}{p_\theta(x)}\, dx = \mathbb{E}\left[f(x)\, \nabla_\theta \log p_\theta(x)\right]$$
- The gradient of an **expectation** becomes an **expectation of a product** — which we can estimate by **sampling**!

13

The Policy Gradient Theorem

- Apply the log-derivative trick to the RL objective:
- $$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}\left[\sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t|s_t) \cdot G_t\right]$$
- Where $G_t = \sum_{k=0}^{T-t} \gamma^k r_{t+k}$ is the **return** from timestep t

14

Intuition: What Does It Mean?

- $\nabla_\theta \log \pi_\theta(a_t|s_t)$: the direction that makes action a_t **more probable**
- G_t: how good the **outcome** was after taking that action
- **The product**: if the outcome was **good** (G_t large), push parameters to make that action more likely
- If the outcome was **bad** (G_t small or negative), push parameters away
- This is a **reward-weighted** version of the cross-entropy gradient you know from SFT!
- In SFT, every token is equally important. In RL, tokens that led to **high reward get upweighted**

15

The REINFORCE Algorithm

- The simplest policy gradient algorithm (Williams, 1992):

REINFORCE

```
for each training iteration:
    # 1. Sample trajectory using current policy
    trajectory = rollout(π_θ)

    # 2. Compute returns for each timestep
    G = []
    running_return = 0
    for t in reversed(range(len(trajectory))):
        running_return = r[t] + γ * running_return
        G.insert(0, running_return)

    # 3. Policy gradient update
    loss = -sum(log π_θ(a_t|s_t) * G[t] for t)
    loss.backward()
    optimizer.step()
```

16

The Problem: High Variance

- Suppose two trajectories both succeed, scoring $G = 100$ and $G = 102$
- REINFORCE says: make **both** more likely! But one is barely better
- Or suppose **all rewards are positive** — every action gets reinforced, regardless of quality
- The gradient estimate is **unbiased** but has **enormous variance**
- **High variance** → slow, unstable learning → we need to fix this

17

Part 3: Variance Reduction

18

Interactive Demo: Advantage & Variance

🚀 **Interactive Demo:** advantage_demo.html

19

Baselines: A Simple Fix

- Subtract any function $b(s_t)$ that depends **only on the state** (not the action):
- $$\nabla_\theta J = \mathbb{E}\left[\sum_t \nabla_\theta \log \pi_\theta(a_t|s_t) \cdot (G_t - b(s_t))\right]$$
- **Why is this still unbiased?** Because:
- $$\mathbb{E}_{a\sim\pi_\theta}[\nabla_\theta \log \pi_\theta(a|s) \cdot b(s)] = b(s) \cdot \nabla_\theta \underbrace{\sum_a \pi_\theta(a|s)}_{=1} = 0$$
- The optimal baseline is close to $V^\pi(s_t)$ — the **expected return from state** s_t
- Now we ask: was this trajectory **better or worse than expected?**

20

The Advantage Function

- $$A^\pi(s,a) = Q^\pi(s,a) - V^\pi(s)$$
- $A > 0$: this action was **better than average** for this state → increase probability
- $A < 0$: **worse than average** → decrease probability
- $A = 0$: exactly as expected → no change
- The advantage asks: *was this action surprisingly good or surprisingly bad, given where I was?*
- Much **cleaner signal** than raw return

21

Actor-Critic Architecture

- Use **two** neural network components:
- **Actor** $\pi_\theta(a|s)$: the policy network — decides **what to do**
- **Critic** $V_\phi(s)$: the value network — estimates **how good the current state is**
- The critic provides the baseline to compute advantages
- Simplest advantage estimate — **one-step TD error**:
- $$\hat{A}_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$$
- Low variance (single transition) but **biased** (relies on critic accuracy)

22

Generalized Advantage Estimation (GAE)

- GAE gives us a **dial** between bias and variance:
- $$\hat{A}_t^{\text{GAE}} = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}$$
- Where $\delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$ is the **TD error**
- $\lambda = 0$: use only δ_t — low variance, higher bias
- $\lambda = 1$: sum all future TD errors — zero bias, high variance (Monte Carlo)
- $\lambda \approx 0.95$: the **practical sweet spot** used in all modern implementations

23

The λ Bias-Variance Tradeoff

- λ lets you continuously interpolate:
- **Low** λ: trust the critic more (good when critic is accurate)
- **High** λ: trust the actual returns more (good early in training when critic is bad)
- As training progresses and the critic improves, lower λ reduces noise
- This is exactly the same bias-variance tradeoff from ML — but applied to **credit assignment** in RL

24

Part 4: Mapping to LLMs

25

Actor-Critic for Language Models

- **Actor**: the transformer → $\pi_\theta(\text{next token}|\text{prompt} + \text{tokens so far})$
- **Critic**: same transformer backbone with a **scalar value head** → $V_\phi(\text{context}) \in \mathbb{R}$
- Both trained simultaneously during RL
- Critic loss: $L_{\text{value}}(\phi) = \mathbb{E}_t \left[(V_\phi(s_t) - G_t)^2\right]$
- The critic learns to predict how much total reward will come from the current context

26

RL Concepts → LLM Training

RL Concept	LLM Equivalent
Policy $\pi_\theta(a\|s)$	Transformer softmax over vocabulary
Trajectory τ	One complete generated response
Return G_t	Score assigned to the response
Advantage $\hat{A}_t$	Was this token better/worse than expected?
Actor	Transformer (language model head)
Critic	Transformer (scalar value head)
Policy gradient	Reward-weighted cross-entropy loss

27

Summary

28

All Interactive Demos

🚀 **Interactive Demo:** policy_gradient_demo.html

🚀 **Interactive Demo:** advantage_demo.html

29

Lecture Summary

- **Value-based RL** (DQN) requires $\max$ over all actions — impractical for 100K-token vocabularies
- **Policy Gradient Theorem**: $\nabla_\theta J = \mathbb{E}[\nabla_\theta \log \pi_\theta(a|s) \cdot G_t]$ — optimize the policy directly
- **REINFORCE**: simple but high variance — every trajectory gets reinforced
- **Advantage function**: $A = Q - V$ asks 'better or worse than expected?' — much cleaner signal
- **Actor-Critic + GAE**: neural critic provides baseline, λ controls bias-variance tradeoff
- **Next**: How far do we step? One bad update can collapse a billion-parameter model → **PPO**

30

Supplementary Resources

🚀 **Interactive Demo:** handout.html

31